SonicBoom: 第三次アウトオブオーダプロセッサ

Jerry Zhao, Ben Korpan, Abraham Gonzalez, Krste Asanovic

英語論文購読第２回　ヴハイナム

Table of Content

Introduction

- Deployment of high-performance superscalar, out-of-order (OOO) cores expanding into mobile and edge devices
- Security, power, performance, area need to be considered when evaluating new design
- An open-source hardware implementation of a superscalar, OOO core is invaluable

- Open source hardware implementation have numerous advantages over software models of high-performance cores
  - Demonstrate precise microarchitectural behaviors
  - Execute real applications for trillions of cycles
  - *Empirically* provide power and area measurements
  - Point of comparison for new microarchitectural platform

- Most of the current open-source hardware development framework only support simple in-order cores
  - Rocket
  - Ariane
  - Black Parrot
  - PicoRV32
- Modern mobile / server-class SOC is impossible without a full-featured, high performance implementation of a superscalar OOO core.

***Fastest* open-source** core

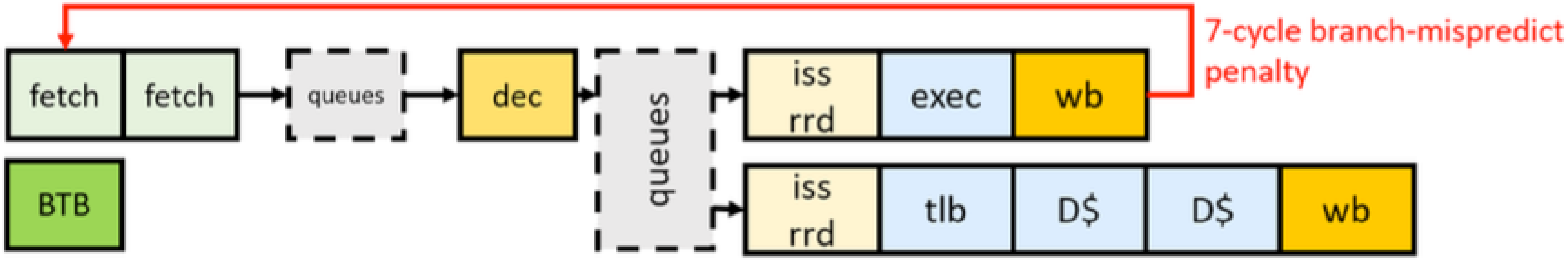
Table 1: Comparison of open-source out-of-order cores.

	***Sonic* BOOM**	**BOOMv2**	**riscy-OOO**	**RSD**
ISA	RV64GC	RV64G	RV64G	RV32IM
DMIPS /MHz	3.93	1.91	?	2.04
SPEC06 IPC	0.86	0.42	0.48	N/A
Branch Predictor	TAGE	GShare	Tourney	GShare
Dec Width	1-5	1-4	2	2
Mem Width	16b/cycle	8b/cycle	8b/cycle	4b/cycle
HDL	Chisel3	Chisel2	BSV+CMD	System Verilog

BOOM history

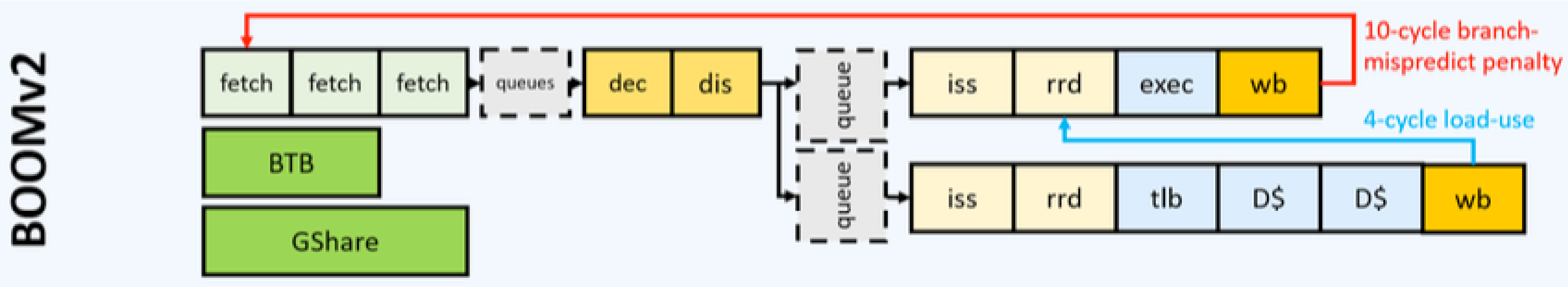
- BOOM Version 1
  - Education tool for University
  - Based on microprocessor MIPS R10K (A RISC implementation of MIPS IV ISA)
  - Not enough pipeline stages, thus **not physically realisable**

BOOMv1

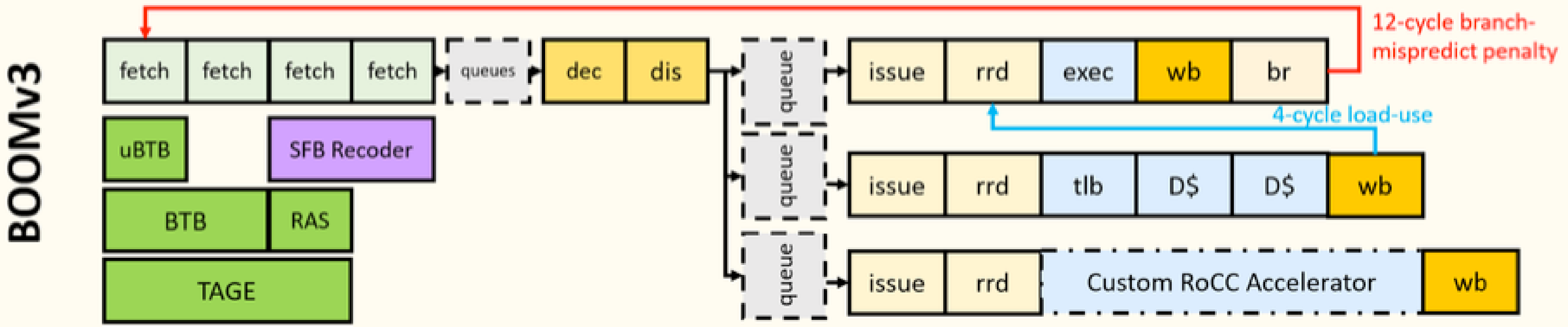


fetch
fetch
queues
dec
queues
iss
rrd
exec
wb
7-cycle branch-mispredict penalty
BTB
iss
rrd
tlb
D$
D$
wb

- BOOM Version 2
  - Add more pipeline to Front-end and Execution paths
  - Split floating-point register file and execution units to independent pipeline
  - Split issue queues to separate queues: Int, Memory, Float operations


BOOMv2
fetch
fetch
fetch
queues
dec
dis
queue
queue
iss
rrd
exec
wb
iss
rrd
tlb
D$
D$
wb
BTB
GShare
10-cycle branch-mispredict penalty
4-cycle load-use

- Sonic BOOM
  - Support more software stacks
  - Fix performance bottlenecks, keep physical realizability
    - IF
    - EX backend
    - Load / Store
  - Improved critical path delay


BOOMv3
fetch
fetch
fetch
fetch
queues
dec
dis
queue
queue
queue
issue
rrd
exec
wb
br
12-cycle branch-
mispredict penalty
4-cycle load-use
issue
rrd
tlb
D$
D$
wb
issue
rrd
Custom RoCC Accelerator
wb
uBTB
SFB Recoder
BTB
RAS
TAGE

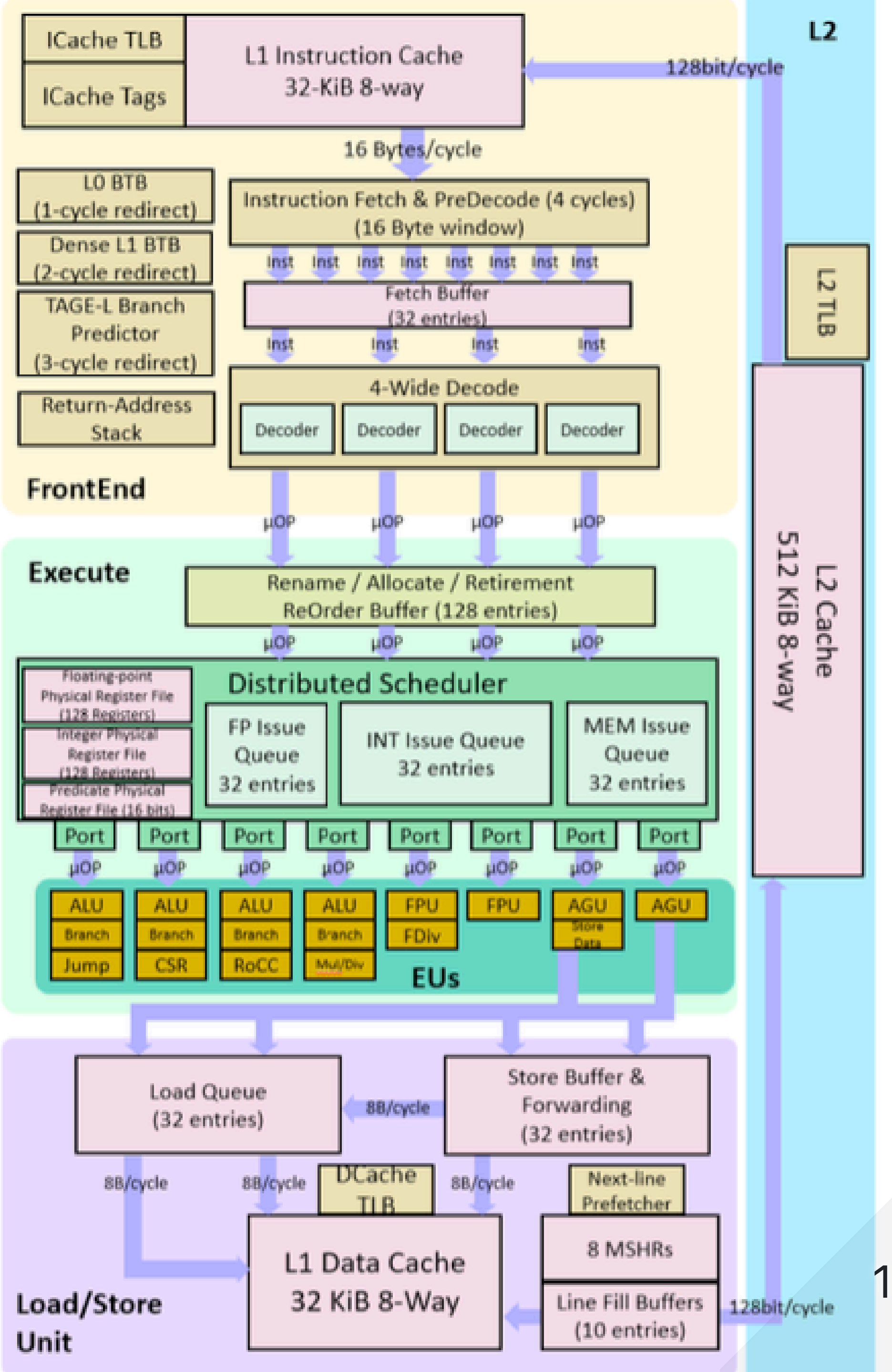
- In short
  - If BOOMv1 was created for educational purpose BOOMv3 (SonicBOOM) was created for research in hi-performance core design

Instruction Fetch

- Support for 2-byte RVC instructions
- Implementation of TAGE branch predictor
- Multiple branch-resolution units

Support for 2-byte forms of common 4-byte instructions

- Default RV-ISA subset for packaged Linux distributions (Fedora, Debian)
- New fetch-unit which decodes possible 2 / 4 bytes instruction for every 2 bytes


L2
ICache TLB
ICache Tags
L1 Instruction Cache
32-KiB 8-way
128bit/cycle
16 Bytes/cycle
L0 BTB
(1-cycle redirect)
Dense L1 BTB
(2-cycle redirect)
TAGE-L Branch
Predictor
(3-cycle redirect)
Return-Address
Stack
Instruction Fetch & PreDecode (4 cycles)
(16 Byte window)
Inst
Fetch Buffer
(32 entries)
4-Wide Decode
Decoder
FrontEnd
L2 TLB
L2 Cache
512 KiB 8-way
μOP
Execute
Rename / Allocate / Retirement
ReOrder Buffer (128 entries)
Distributed Scheduler
Floating-point
Physical Register File
(128 Registers)
Integer Physical
Register File
(128 Registers)
Predicate Physical
Register File (16 bits)
FP Issue
Queue
32 entries
INT Issue Queue
32 entries
MEM Issue
Queue
32 entries
Port
ALU
Branch
Jump
CSR
RoCC
Mul/Div
FPU
FDiv
AGU
Store
Data
EUs
Load Queue
(32 entries)
8B/cycle
Store Buffer &
Forwarding
(32 entries)
DCache
TLB
Next-line
Prefetcher
L1 Data Cache
32 KiB 8-Way
8 MSHRs
Line Fill Buffers
(10 entries)
128bit/cycle
Load/Store
Unit